OPERATING SYSTEMS: PROCESS SYNCHRONIZATION AND DEADLOCK MANAGEMENT

1. Introduction to Process Synchronization Process synchronization is defined as the coordination of simultaneous execution of processes such that they access shared system resources without data inconsistency. In multiprogramming systems, concurrent access to shared data may result in data inconsistency. Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.

2. The Critical Section Problem A critical section refers to a segment of code in which a process accesses shared resources, such as common variables, files, or hardware tables. When one process is executing in its critical section, no other process is allowed to execute in its critical section. A complete solution to the critical section problem must satisfy three essential criteria: First, Mutual Exclusion: If process Pi is executing in its critical section, then no other processes can be executing in their critical sections. Second, Progress: If no process is executing in its critical section and some processes wish to enter their critical sections, only those processes not executing in their remainder sections can participate in deciding which will enter next, and this selection cannot be postponed indefinitely. Third, Bounded Waiting: There must be a bound or limit on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter and before that request is granted.

3. Synchronization Hardware and Software Primitives Peterson's Algorithm is a software-based solution to the critical section problem restricted to two processes. It utilizes two shared variables: an integer variable 'turn' indicating whose turn it is to enter the critical section, and a boolean array 'flag' indicating whether a process is ready to enter.

A Semaphore is defined as a synchronization tool that provides a sophisticated way for processes to coordinate their activities. An integer semaphore S is accessed only through two standard atomic operations: wait() and signal(). The wait() operation decrements the semaphore value, while the signal() operation increments the semaphore value. Semaphores are categorized into two types: Binary Semaphores: The value of a binary semaphore can range only between 0 and 1. They are also known as mutex locks because they provide mutual exclusion. Counting Semaphores: The value of a counting semaphore can range over an unrestricted domain. They are used to control access to a given resource consisting of a finite number of instances.

4. The Deadlock Problem In a multiprogramming environment, several processes may compete for a finite set of resources. A process requests resources; if the resources are not available at that time, the process enters a waiting state. A Deadlock is defined as a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process in the same set.

5. Necessary Conditions for Deadlock Coffman established that a deadlock can arise if and only if the following four conditions hold simultaneously in a system: First, Mutual Exclusion: At least one resource must be held in a non-sharable mode. Only one process can use the resource at any given time.

Second, Hold and Wait: A process must be currently holding at least one resource and simultaneously waiting to acquire additional resources that are currently being held by other processes. Third, No Preemption: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task. Fourth, Circular Wait: A closed chain of processes exists such that each process holds at least one resource needed by the next process in the chain.

6. Deadlock Handling Strategies Operating systems typically handle deadlocks using three main methodologies: Deadlock Prevention: Provides a set of methods to ensure that at least one of the four necessary conditions cannot hold. For instance, preventing Hold and Wait requires processes to request all resources at once. Deadlock Avoidance: Requires that the operating system be given in advance additional information concerning which resources a process will request and use during its lifetime. With this knowledge, the system can decide for each request whether or not the process may wait. The Banker's Algorithm is a deadlock avoidance algorithm developed by Edsger Dijkstra. It tests for safety by simulating the allocation of predetermined maximum possible amounts of all resources, and then makes an s-state check to test for possible activities before deciding whether allocation should be allowed to continue. Deadlock Detection and Recovery: Allows the system to enter a deadlock state, detects the occurrence using wait-for graphs, and recovers by process termination or resource preemption.